

FIL GURGAON · DEVELOPER ENABLEMENT

Jenkins

Hands-On

CI/CD pipelines for Python developers — from an empty laptop to a deploying banking service

Self-learning

Local Jenkins in Docker

Python only

6 labs + capstone

Read it, follow it, run it — you'll finish with a working pipeline.

Pipeline #42

banking-api · main

SUCCESS



Checkout

2s



Install deps

14s



Run tests

6s



Build image

21s



Deploy

9s



Total 58s · triggered by git push

READ ME FIRST

How to use this deck

Work top-to-bottom. These labels tell you what to do:

DO THIS

Hands-on action — type or click exactly as shown.

CHECKPOINT

Confirm you see this before moving on.

TIP

A shortcut or a good practice.

WATCH OUT

A common mistake — and its fix.

QUICK CHECK

Test yourself — the answer is on the slide.

ARCHITECTURE

Follow the numbered markers with the legend.



Dark boxes are code windows — copy-paste ready. Blue-grey lines are comments.

TWO ENVIRONMENTS



Your local Jenkins

ALL LABS

- Runs in Docker on your laptop
- Opens at <http://localhost:8080>
- You are the admin — break it freely



FIL shared Jenkins

AWARENESS ONLY

- Managed by the platform team
- Log in with your FIL SSO
- Used for real projects after training

OUTCOMES

What you'll be able to do by the end



Run your own Jenkins

Start Jenkins in Docker and complete the setup wizard.



Write a Jenkinsfile

Read and author a Declarative pipeline — stages, steps, post.



Build & test Python

A job that creates a venv, installs deps and runs pytest.



Use Docker agents

Run builds inside clean, disposable Python containers.



Handle secrets safely

Store an API key in Jenkins and use it without leaking it.



Ship a banking service

Commit → acceptance → release with one approval gate.

Nine modules, one working pipeline

1 Foundations

CI/CD & Jenkins architecture

🕒 30 min

2 Set up your Jenkins

Lab 0 — Jenkins in Docker

🕒 30 min

3 Pipeline as Code

Anatomy of a Jenkinsfile

🕒 20 min

4 First Python pipeline

Lab 1 — build & test

🕒 30 min

5 Agents & Docker

Lab 2 — containerised builds

🕒 20 min

6 Secrets

Lab 3 — inject an API key safely

🕒 20 min

7 Reuse & governance

Shared libraries & security gates

🕒 30 min

8 Capstone

Ship a Python banking API

🕒 60 min

9 Toolkit

Debugging, glossary, next steps

🕒 15 min



MODULE 1

Foundations

What CI/CD means, and how Jenkins is put together — before we touch a keyboard.

01

IN THIS MODULE

- ✓ Why CI/CD matters in banking
- ✓ End-to-end CI/CD architecture
- ✓ Inside Jenkins: controller, agents, executors
- ✓ The life of a single build
- ✓ How Jenkins runs at FIL (awareness)



Why automate? A banking reality check

Manual releases

-  "Works on my machine" — each developer builds differently.
-  Deploy steps typed by hand at 11 pm — one gets missed.
-  No record of what was tested, by whom, with which config.
-  Bugs found in production — customer money at risk.

With a Jenkins pipeline

-  The same automated build runs on every push.
-  Scripted, repeatable steps — nothing is skipped.
-  Every run is logged: who, what, results, artifacts.
-  Bugs caught minutes after the commit.

Jenkins is an open-source automation server that runs your build → test → deploy steps the same way, every time.



Why Jenkins fits a bank like FIL



Self-hosted, in your control

Runs inside FIL's own network. Code and secrets never leave the bank — important for compliance.



Pipeline as Code

Pipelines live in a Jenkinsfile in Git — versioned, peer-reviewed and auditable like application code.



Huge plugin ecosystem

Git, Docker, SonarQube, Jira, Kubernetes... almost any integration already exists as a plugin.



Mature & battle-tested

10+ years in large enterprises. Well documented, widely known, fine-grained permissions.

NOTE

GitHub Actions and GitLab CI use the same ideas — pipelines, stages, runners/agents, secrets. Everything you learn here transfers.



A pipeline is a chain of stages



Continuous Integration

Merge small changes often. Every push is built and tested automatically.



Continuous Delivery

Every green build is releasable. A human approves the production release.

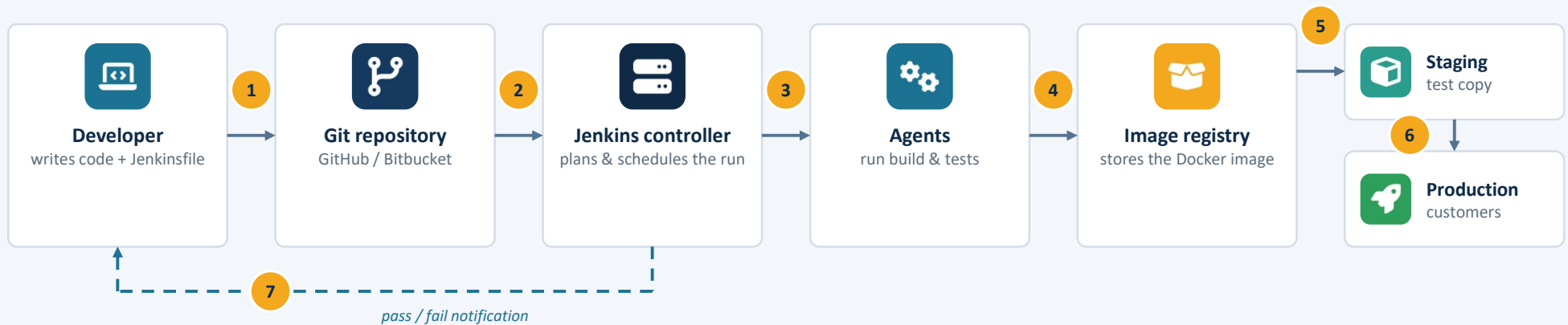
TYPICAL IN BANKING



Continuous Deployment

Every green build goes to production automatically — no human gate.

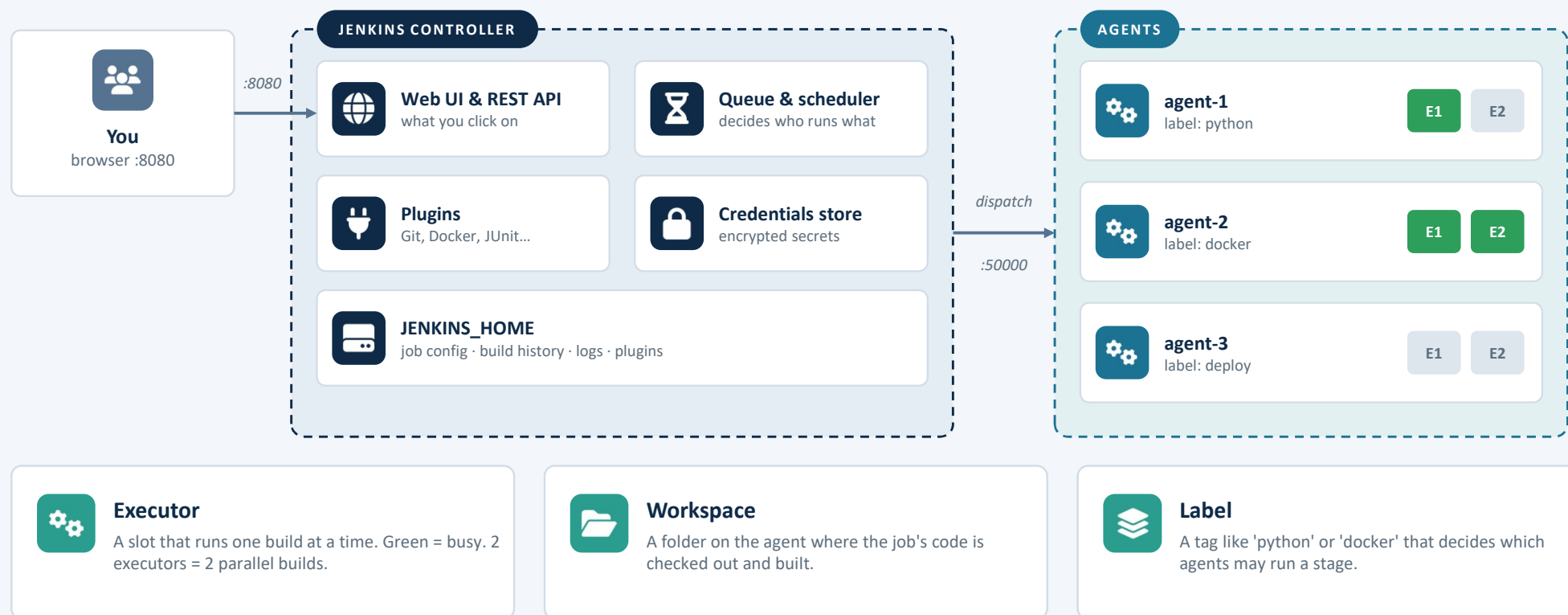
How a change travels from laptop to production



HOW IT FLOWS

- 1 You push code (and the Jenkinsfile) to Git.
- 2 Git notifies Jenkins with a webhook — a new run starts.
- 3 The controller hands the work to a free agent.
- 4 The agent builds, tests and pushes a Docker image.
- 5 The image is deployed to staging and smoke-tested.
- 6 After approval, the SAME image is promoted to production.
- 7 Jenkins reports pass / fail back to the team.

Controller, agents and executors

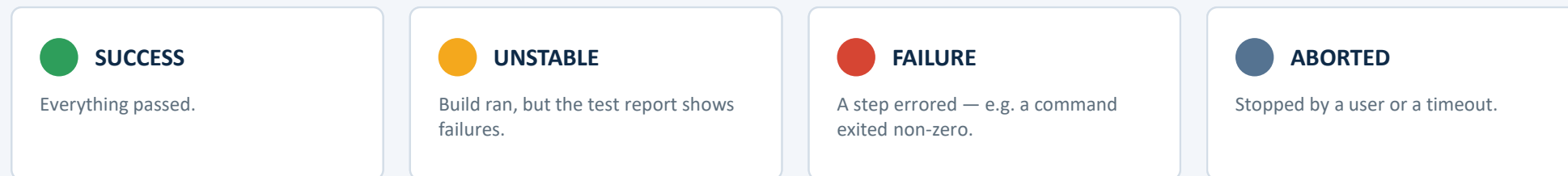


Golden rule: the controller orchestrates, agents execute. Today your one container does both — at FIL, builds run only on agents.

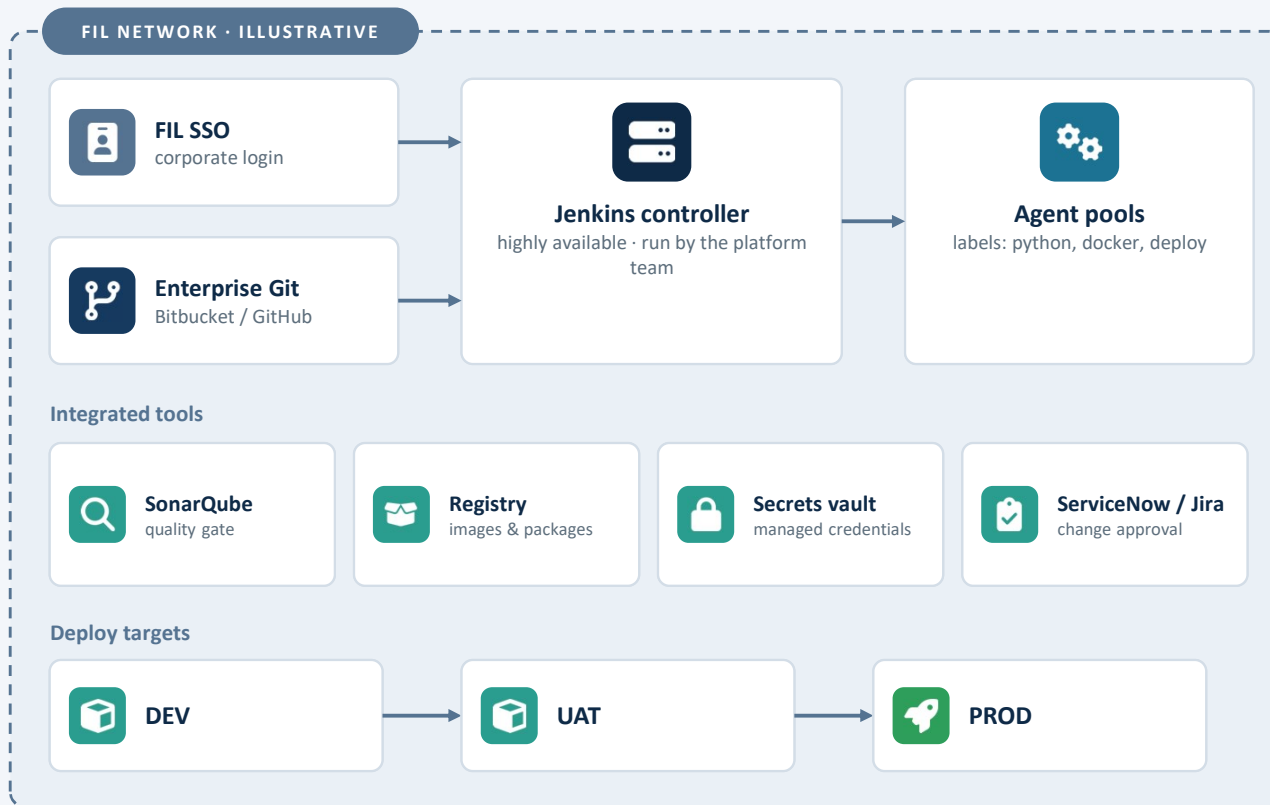
The life of a single build



BUILD RESULTS YOU'LL SEE



How Jenkins typically runs at FIL



What's different for you

- Log in with SSO — no setup, no admin password.
- Target agents by label: agent { label 'python' }.
- Secrets already exist — you only use their IDs.
- Common steps come from a shared library.
- Quality gates & approvals are mandatory and audited.

Exact tools and names may differ — ask the platform team.



MODULE 2

Set up your Jenkins

Run a complete Jenkins on your own laptop with Docker — this is where every lab happens.

IN THIS MODULE

- ✓ Prerequisites
- ✓ Your lab architecture
- ✓ Lab 0 — build, run & unlock Jenkins
- ✓ Lab 0 — add the Docker plugin + smoke test
- ✓ A tour of the screens you'll use

02



Prerequisites checklist

**Docker Desktop, running**

8 GB+ RAM laptop; give Docker about 4 GB

**Git**

to push your lab repositories

**GitHub or Bitbucket account**

free account; public repos are fine for labs

**Python 3.8+**

only for editing / running code locally

**A code editor**

VS Code, PyCharm — any

**About 5 GB free disk**

Jenkins + Python images

Terminal

```
# Verify your laptop
docker --version    # Docker version 24+
docker info         # must NOT say "Cannot connect"
git --version
python --version    # Python 3.8+
```

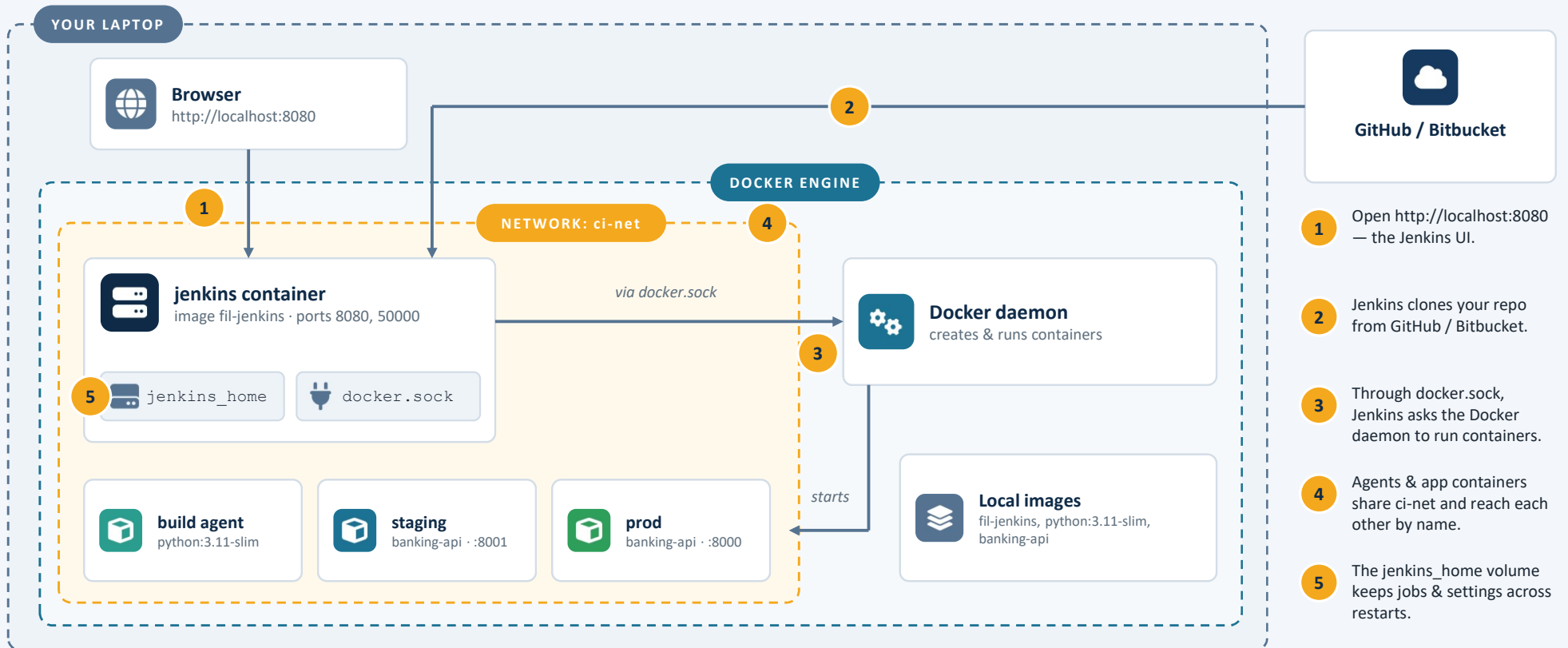
WATCH OUT

Corporate laptop? Docker Desktop may need IT approval and a licence — request it before the session.

**TIP**

Windows: use Docker Desktop with the WSL 2 backend, and run the lab commands in Git Bash or a WSL terminal.

What you're about to build on your laptop



Build & run your Jenkins container

DO THIS

Make a folder jenkins-lab, save the Dockerfile in it, then run the terminal commands.

10 min

Dockerfile

```
# Official Jenkins + Python + Docker CLI
FROM jenkins/jenkins:lts-jdk17
USER root
RUN apt-get update && \
    apt-get install -y python3 python3-pip \
        python3-venv docker.io && \
    ln -sf /usr/bin/python3 /usr/bin/python && \
    rm -rf /var/lib/apt/lists/*
USER jenkins
```

TIP

The jenkins_home volume keeps all Jenkins data, even if you delete and re-create the container.

WATCH OUT

Windows PowerShell: replace each trailing \ with a backtick (`), or run the commands in Git Bash / WSL.

Terminal

```
# 1) Create the lab network (once)
docker network create ci-net

# 2) Build the image (same folder as Dockerfile)
docker build -t fil-jenkins .

# 3) Start Jenkins
docker run -d --name jenkins --network ci-net \
    -p 8080:8080 -p 50000:50000 \
    -v jenkins_home:/var/jenkins_home \
    -v /var/run/docker.sock:/var/run/docker.sock \
    fil-jenkins

# 4) Is it running?
docker ps --filter name=jenkins
```

CHECKPOINT

docker ps shows a container named jenkins with STATUS “Up ...”.



Unlock Jenkins & finish the setup wizard

DO THIS

Open your browser and follow the wizard.

10 min

1 Open the UI

Browse to <http://localhost:8080>

2 Paste the admin password

Get it with the command on the right.

3 Install suggested plugins

Wait 2–5 minutes. If one fails, click Retry.

4 Create your admin user

Pick a username and password you'll remember.

5 Keep the Jenkins URL

Leave <http://localhost:8080/> → Save and Finish.

Terminal

```
# Reveal the one-time admin password
docker exec jenkins cat \
  /var/jenkins_home/secrets/initialAdminPassword

# Still booting? Follow the log until you see
# "Jenkins is fully up and running"
docker logs -f jenkins
```

CHECKPOINT

You land on the dashboard showing “Welcome to Jenkins!”. Keep this container running for the whole workshop.

WATCH OUT

Behind a corporate proxy? Set it under Manage Jenkins → Plugins → Advanced settings, then retry.

Add the Docker plugin & run a smoke test

DO THIS

Install one plugin, then prove Python and Docker both work inside Jenkins.

🕒 10 min

1 Manage Jenkins → Plugins

Open the Available plugins tab.

2 Install “Docker Pipeline”

Search, tick it, click Install. No restart needed.

3 New Item → smoke-test → Pipeline

Definition: Pipeline script → paste the script on the right → Save.

4 Build Now → #1 → Console Output

Compare with the expected output.

⚠️ WATCH OUT

“permission denied ... docker.sock”? Lab-only fix: `docker exec -u root jenkins chmod 666 /var/run/docker.sock` (repeat after Docker restarts).

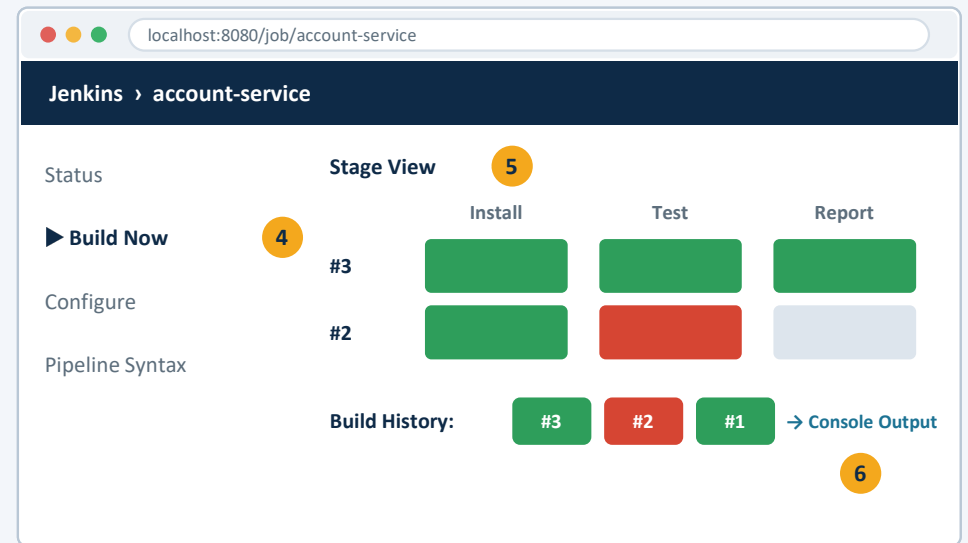
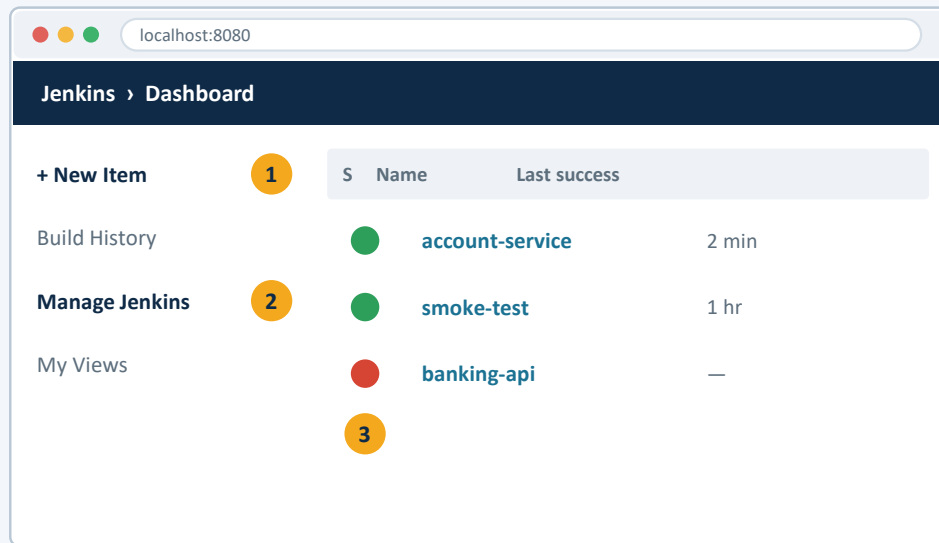
🔴 🟡 🟢 Pipeline script

```
pipeline {
  agent any
  stages {
    stage('Smoke test') {
      steps {
        sh 'python --version'
        sh 'docker version --format "{{.Server.Version}}"'
        sh 'docker ps'
      }
    }
  }
}
```

🔴 🟡 🟢 Console Output — expected

```
+ python --version
Python 3.11.2
+ docker version --format "{{.Server.Version}}"
27.3.1
+ docker ps
CONTAINER ID   IMAGE      ...   NAMES
alb2c3d4e5f6   fil-jenkins ...   jenkins
Finished: SUCCESS
```

The Jenkins screens you'll use



1 New Item — create a job (pick Pipeline).

2 Manage Jenkins — plugins, settings, Credentials.

3 Status — green = success, red = failed.

4 Build Now — starts a run (#1, #2, ...).

5 Stage View — which stage failed, and how long each took.

6 Click a build → Console Output: the full log. Read it first.



MODULE 3

Pipeline as Code

The Jenkinsfile — one text file in Git that describes your whole pipeline.

IN THIS MODULE

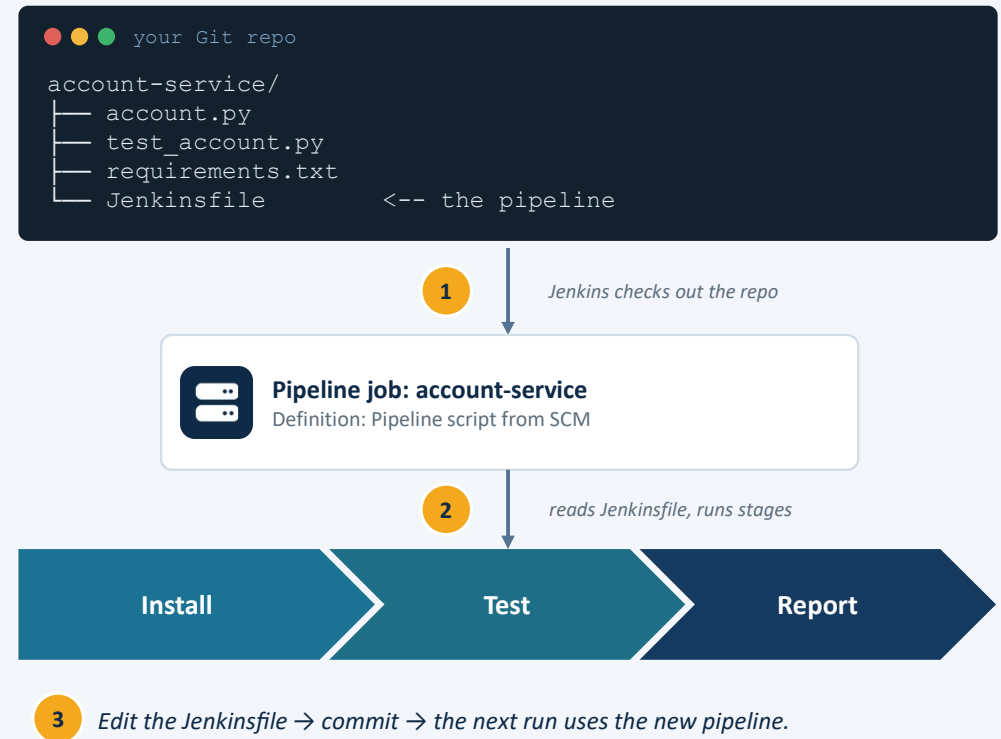
- ✓ What a Jenkinsfile is — and where it lives
- ✓ The 5 parts of a Declarative pipeline
- ✓ Directives cheat sheet

03

What is a Jenkinsfile?

Your pipeline, written as code

-  A text file named Jenkinsfile in the repo root.
-  Describes every stage: install, test, package, deploy.
-  Versioned in Git with your code — same branch, same pull request.
-  Readable: anyone can see exactly what runs.
-  Auditable: every change to the pipeline has an author and a review.



The 5 parts of a Declarative Jenkinsfile

```
Jenkinsfile

pipeline {                                // 1 the whole pipeline
  agent any                               // 2 WHERE it runs
  environment {                           // 3 variables for all stages
    APP = 'account-service'
  }
  stages {                                // 4 the ordered work
    stage('Build') {
      steps {
        echo "Building ${APP}"
        sh 'python --version'
      }
    }
  }
  post {                                  // 5 runs at the end
    success { echo 'All good' }
    failure { echo 'Check the log' }
  }
}
```

**TIP**

Online you'll also see Scripted pipelines (node { ... }). We use Declarative: easier to read, and validated before it runs.

1**pipeline**

The outer block. Everything lives inside it.

2**agent**

WHERE the work runs — any, a label, or a Docker image.

3**environment**

Key-value variables available to every stage.

4**stages / steps**

Ordered stages; each has steps (the commands).

5**post**

Runs after the stages — on success, failure or always.

Directives you'll use again and again

Directive	What it does	Example
agent	Where a stage runs	<code>agent { docker { image 'python:3.11-slim' } }</code>
environment	Variables for the run	<code>environment { APP = 'account-service' }</code>
options	Run-wide settings	<code>options { timeout(time: 20, unit: 'MINUTES') }</code>
triggers	Start runs automatically	<code>triggers { pollSCM('H/5 * * * *') }</code>
when	Run a stage only if... (multibranch)	<code>when { branch 'main' }</code>
sh	Run a shell command	<code>sh 'pytest -v'</code>
withCredentials	Inject a secret	<code>withCredentials([string(...)]) { ... }</code>
input	Pause for human approval	<code>input message: 'Deploy to prod?'</code>
post	React to the result	<code>post { failure { echo 'broke' } }</code>



MODULE 4

First Python pipeline

Lab 1 — Jenkins checks out a Python service, installs it and runs its tests automatically.

IN THIS MODULE

- ✓ The lab brief
- ✓ Create the sample project
- ✓ Write the Jenkinsfile
- ✓ Create & run the job, click by click
- ✓ Read the result — then break it on purpose

04

Build & test a Python service automatically



Goal

Jenkins checks out your repo, installs dependencies in a venv, runs pytest and publishes the test report — on every run.



You need

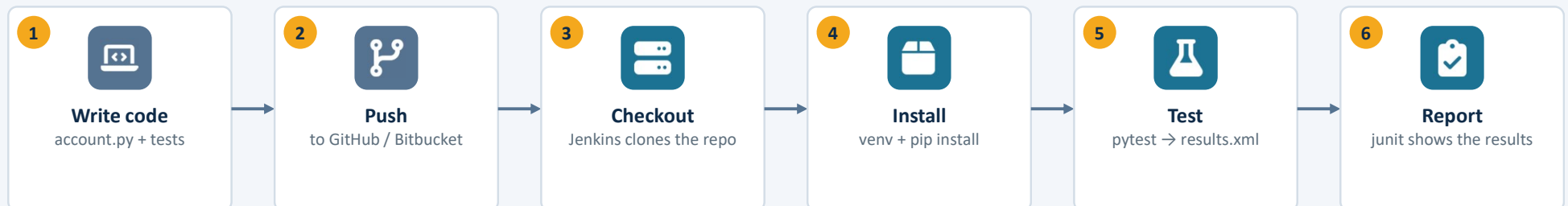
Jenkins from Lab 0 (smoke test passed), a GitHub/Bitbucket account, and about 30 minutes.



Done when

Run #1 ends in SUCCESS, Test Result shows 2 passed, and you've seen — and fixed — a red run.

WHAT HAPPENS



Create the sample Python project

DO THIS

Make a folder `account-service` with these files, then push it to a new Git repo.

🕒 10 min

account.py

```
# account.py
def account_balance(deposits, withdrawals):
    """Net balance for an account."""
    return sum(deposits) - sum(withdrawals)
```

test_account.py

```
# test_account.py
from account import account_balance

def test_positive():
    assert account_balance([100, 50], [30]) == 120

def test_zero():
    assert account_balance([], []) == 0
```

requirements.txt

```
pytest==8.2.0
```

Terminal — publish to Git

```
cd account-service
git init
git add .
git commit -m "Account service + tests"
git branch -M main
git remote add origin <your-repo-url>
git push -u origin main
```

NOTE

Keep the repo public for the lab — or add a GitHub token as a Jenkins credential and select it in the job. The Jenkinsfile (next slide) goes in this same folder.

Write the Jenkinsfile

```
Jenkinsfile

pipeline {
  agent any
  stages {
    stage('Install') {
      steps {
        sh '''
          python -m venv --clear .venv
          . .venv/bin/activate
          pip install -r requirements.txt
        '''
      }
    }
    stage('Test') {
      steps {
        sh '''
          . .venv/bin/activate
          pytest -v --junitxml=results.xml
        '''
      }
    }
    stage('Report') {
      steps { junit 'results.xml' }
    }
  }
  post {
    success { echo 'Account Service passed!' }
    failure { echo 'Failed - open Console Output.' }
  }
}
```

DO THIS

Save as Jenkinsfile (capital J, no extension) in the repo root → commit → push.

TIP

A venv isolates installs from the system Python — and avoids the “externally-managed-environment” error.

NOTE

Jenkins adds a “Checkout SCM” stage automatically before yours — you don't write it.

WATCH OUT

Each sh step is a fresh shell: activate the venv again in every stage that needs it.

Create the Pipeline job, click by click

1 Dashboard → New Item

Name: account-service · type: Pipeline · OK

2 Pipeline section → Definition

Choose “Pipeline script from SCM”.

3 SCM: Git

Paste your repository URL. Credentials: none for a public repo.

4 Branch Specifier

*/main — must match your branch exactly.

5 Script Path

Jenkinsfile (the default) → Save.

6 Build Now → #1 → Console Output

Watch it run.

The screenshot shows the Jenkins configuration page for a new Pipeline job named 'account-service'. The browser address bar shows 'localhost:8080/job/account-service/configure'. The page title is 'Configure > Pipeline'. The configuration fields are as follows:

- Definition:** A dropdown menu set to 'Pipeline script from SCM', marked with a yellow circle containing the number 2.
- SCM:** A dropdown menu set to 'Git', marked with a yellow circle containing the number 3.
- Repository URL:** A text input field containing 'https://github.com/you/account-service.git'.
- Branch Specifier:** A text input field containing '*/main', marked with a yellow circle containing the number 4.
- Script Path:** A text input field containing 'Jenkinsfile', marked with a yellow circle containing the number 5.

At the bottom of the configuration form is a blue 'Save' button.



TIP

Auto-build on every push: add triggers { pollSCM('H/5 * * * *') }. Webhooks can't reach localhost, but polling can.

Read the console, then break it on purpose

Console Output (trimmed)

```
Started by user admin
Obtained Jenkinsfile from git https://github.com/you/...
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] { (Install)
+ python -m venv --clear .venv
+ pip install -r requirements.txt
Successfully installed ... pytest-8.2.0
[Pipeline] { (Test)
+ pytest -v --junitxml=results.xml
test_account.py::test_positive PASSED          [ 50%]
test_account.py::test_zero PASSED              [100%]
===== 2 passed in 0.03s =====
[Pipeline] { (Report)
Recording test results
Account Service passed!
Finished: SUCCESS
```

✓ CHECKPOINT

Finished: SUCCESS, a green status, and the job page shows Test Result: 2 passed.

Stage View after you break it

	Install	Test	Report
#2			
#1			

? QUICK CHECK

Change 120 to 999 in test_positive, push, Build Now. Which stage fails? What is the final status?

Answer: Test fails, Report is skipped, the run ends FAILURE. Revert, push, rebuild → green again.



MODULE 5

Agents & Docker

Run every build inside a clean, disposable container — no more “works on my machine”.

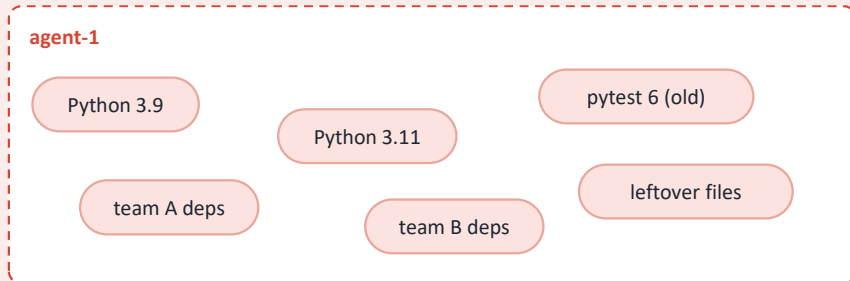
IN THIS MODULE

- ✓ Shared agent vs. Docker agents
- ✓ How a Docker agent works (architecture)
- ✓ Lab 2 — run your tests in python:3.11-slim

05

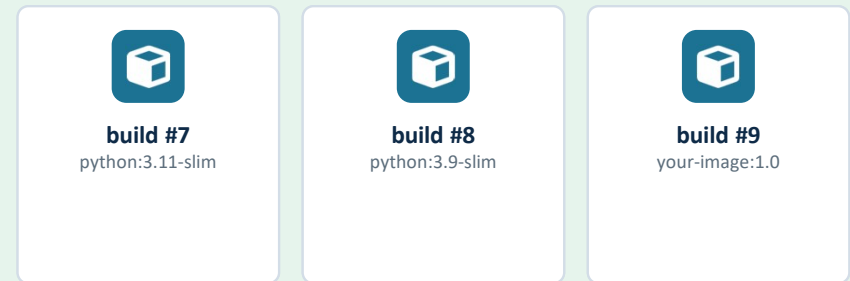
One shared build machine vs. Docker agents

! One shared build machine



- Versions clash between teams.
- Yesterday's leftovers break today's build.
- You can't reproduce last month's build.

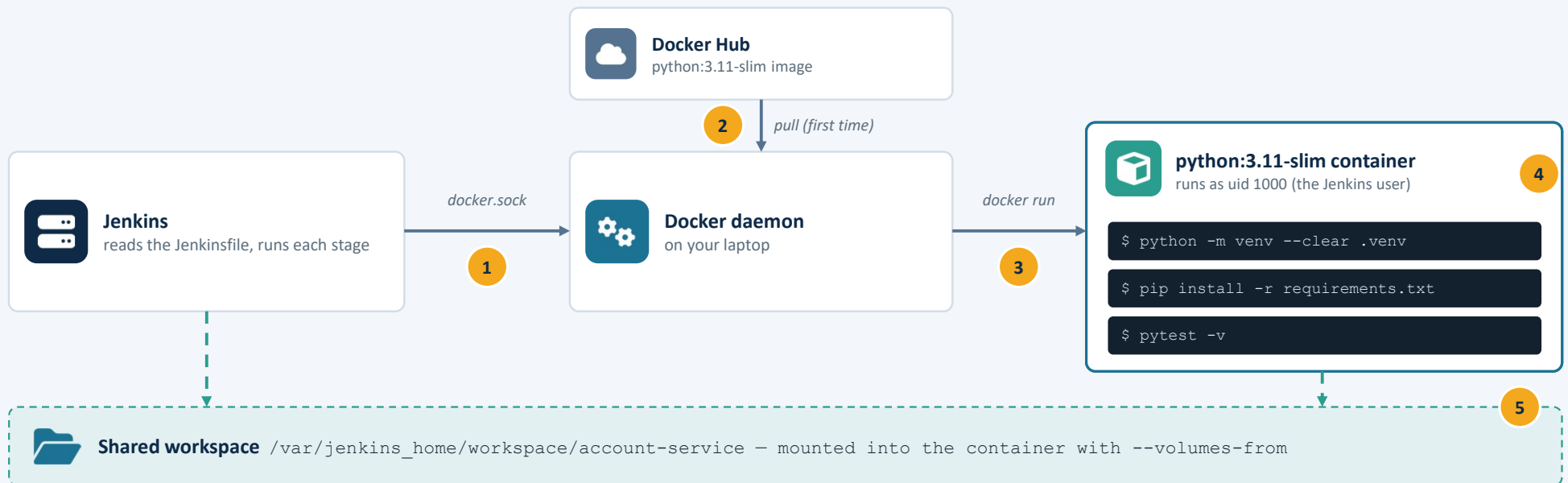
✓ A fresh container per build



- Exact tools per build, declared in the Jenkinsfile.
- Nothing left behind — every run starts clean.
- Same image = same result, today or next month.

At FIL the same idea scales: many Python services on one Jenkins, each build in its own clean image.

What happens when a stage says agent { docker ... }



- 1** The stage starts: Jenkins asks Docker (through docker.sock) for a python:3.11-slim container.
- 2** The image is pulled from Docker Hub — first run only, then cached.
- 3** The container starts as the Jenkins user, sharing the workspace.
- 4** Every sh step in the stage runs inside the container.
- 5** Stage ends → container removed. Files stay in the workspace.

Run your tests inside python:3.11-slim

DO THIS

Replace the Jenkinsfile in account-service with this version, push, then Build Now.

15 min

```
Jenkinsfile

// Jenkinsfile - tests inside a Python container
pipeline {
  agent {
    docker { image 'python:3.11-slim' } // <-- runs here
  }
  stages {
    stage('Test in container') {
      steps {
        sh 'python --version'
        sh '''
        python -m venv --clear .venv
        . .venv/bin/activate
        pip install --no-cache-dir -r requirements.txt
        pytest -v --junitxml=results.xml
        '''
      }
    }
  }
  post {
    always { junit 'results.xml' }
  }
}
```

```
Console Output - look for this

$ docker run -t -d -u 1000:1000 ...
  --volumes-from 3f2a... python:3.11-slim
+ python --version
Python 3.11.x
...
2 passed in 0.04s
Finished: SUCCESS
```

NOTE

-u 1000:1000 means the container runs as the Jenkins user, not root — it can't pip-install into the system Python. That's why we use a venv.

WATCH OUT

“Invalid agent type docker”? Install the Docker Pipeline plugin (Lab 0 · Step 3).



MODULE 6

Secrets

Never hard-code an API key. Store it in Jenkins, inject it at run time, keep it out of the logs.

IN THIS MODULE

- ✓ Credential types & the golden rules
- ✓ How a secret flows (architecture)
- ✓ Lab 3 — add a credential
- ✓ Lab 3 — use it without leaking it

06

Managing secrets the right way

Credential types in Jenkins



Secret text

a token or API key — what we'll use



Username & password

e.g. a registry or database login



Secret file

a whole file, e.g. a kubeconfig



SSH key / certificate

for servers and mutual TLS

Golden rules



Never put a secret in the Jenkinsfile, the code, or Git history.



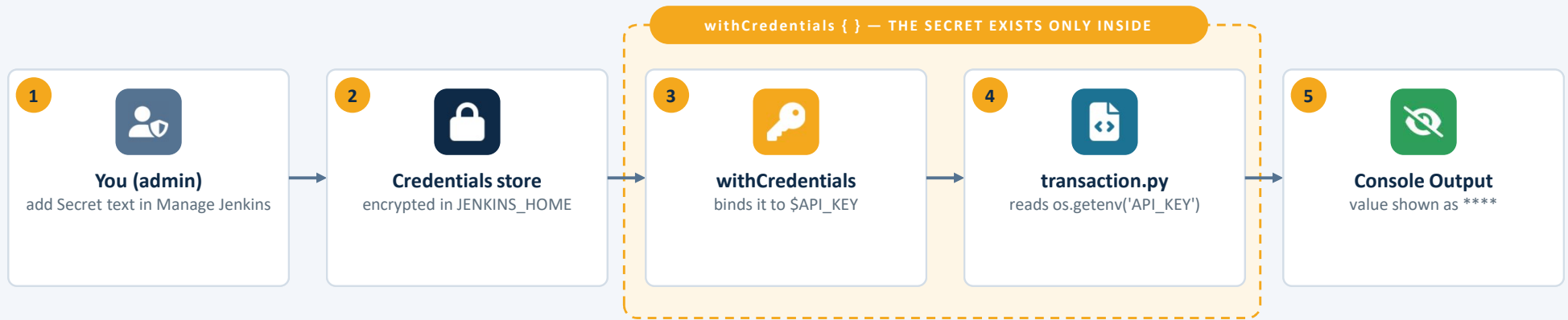
Store it in Jenkins Credentials and reference it by its ID.



Pass it to sh with single quotes — and never print it.

In banking, a leaked key can be a reportable security incident. Treat every credential as production-sensitive.

How a secret travels — and where it stays hidden



✗ Don't — double quotes leak the value into the command

```
leaky

// Groovy expands the secret before the shell runs
sh "python3 transaction.py --key=${API_KEY}"
```

✓ Do — single quotes; the app reads the environment

```
safe

// The shell passes $API_KEY to the process env
sh 'python3 transaction.py'
```

Jenkins even warns you: “A secret was passed to sh using Groovy String interpolation, which is insecure.”

Add the secret in the Jenkins UI

DO THIS

Create one Secret text credential.

🕒 5 min

1 Manage Jenkins → Credentials

Click the (global) domain under “Stores scoped to Jenkins”.

2 Add Credentials

Top-right button.

3 Kind = Secret text

Choose it from the dropdown.

4 Secret

Type a value, e.g. super-secret-123

5 ID

Exactly: transaction-api-key

6 Create

Done — it's stored encrypted.

localhost:8080/manage/credentials/.../newCredentials

New credentials

Kind Secret text 3

Scope Global

Secret 4

ID transaction-api-key 5

Create 6

⚠️ WATCH OUT

The ID must match the Jenkinsfile character-for-character. A typo = “Could not find credentials entry with ID ...”.

Use the secret — without leaking it

DO THIS

Add transaction.py to account-service and this stage to its Jenkinsfile. Push, Build Now.

10 min

```
transaction.py

# transaction.py - reads the key from the environment
import os, sys

api_key = os.getenv("API_KEY")
if not api_key:
    sys.exit("ERROR: API_KEY is not set")

print(f"Loaded API key ({len(api_key)} chars): {'*' * len(api_key)}")
print("Connecting to transaction service...")
```

```
Jenkinsfile - new stage

// add inside stages { } - after 'Test'
stage('Run with Secret') {
    steps {
        withCredentials([string(credentialsId: 'transaction-api-key',
                                variable: 'API_KEY')]) {
            sh 'python3 transaction.py'
        }
    }
}
```

Console Output - expected

```
[Pipeline] withCredentials
Masking supported pattern matches of $API_KEY
+ python3 transaction.py
Loaded API key (16 chars): *****
Connecting to transaction service...
Finished: SUCCESS
```

CHECKPOINT

The key is available as \$API_KEY only inside withCredentials, and Jenkins masks the raw value in the log.

QUICK CHECK

Why does transaction.py print asterisks instead of the key?

Answer: your own code must never log secrets either — Jenkins masking is a safety net, not permission.



MODULE 7

Reuse & governance

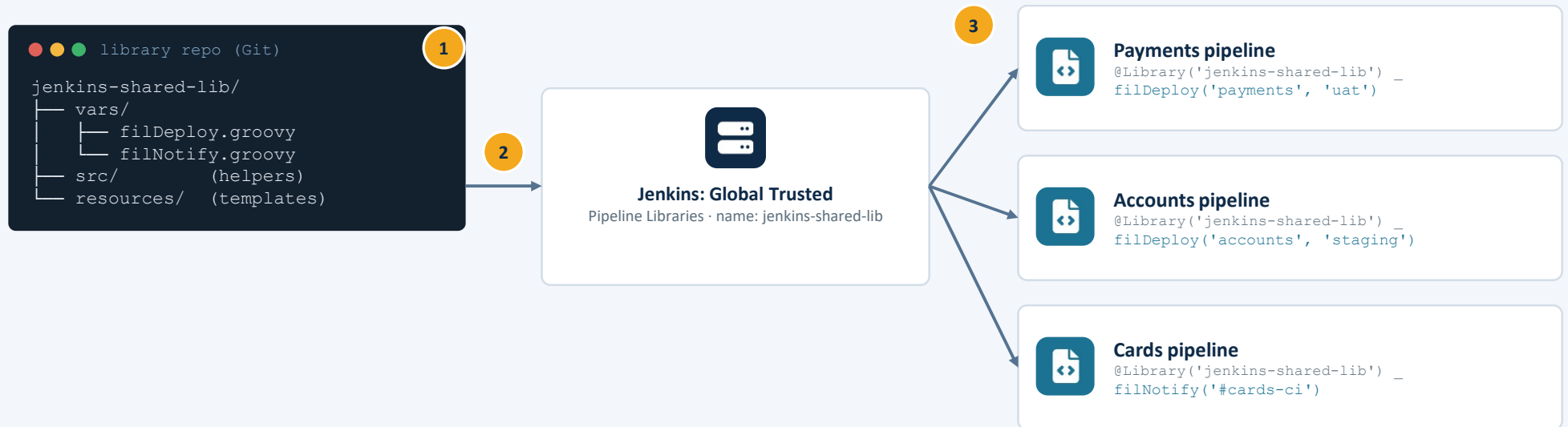
Stop copy-pasting pipelines, and let automated gates block risky code before it ships.

IN THIS MODULE

- ✓ Shared libraries (architecture)
- ✓ Optional try-it: your first library step
- ✓ Where quality & security gates sit
- ✓ Try it: a Python security-scan stage

07

Write a step once — every team reuses it



- 1** Write reusable steps once, in `vars/*.groovy`, in a library repo.
- 2** An admin registers the library in Jenkins — one time.
- 3** Any Jenkinsfile loads it with `@Library` and calls the steps like built-ins.

**TIP**

Fix a bug in the library once → every team gets it on the next run. Pin a version when stability matters: `@Library('jenkins-shared-lib@v1.2') _`

Write your first shared-library step

OPTIONAL

Create a repo jenkins-shared-lib with one file, register it, then call it.

15 min

```
library repo layout

jenkins-shared-lib/
└─ vars/
   └─ filDeploy.groovy
```

```
vars/filDeploy.groovy

// vars/filDeploy.groovy -> new step: filDeploy(...)
def call(String service, String targetEnv) {
    echo "Deploying ${service} to ${targetEnv}..."
    sh "echo '${service} -> ${targetEnv}' > deployment.log"
    archiveArtifacts artifacts: 'deployment.log'
}
```

CHECKPOINT

Console shows “Deploying account-service to staging...” and deployment.log appears under Build Artifacts.

Register once

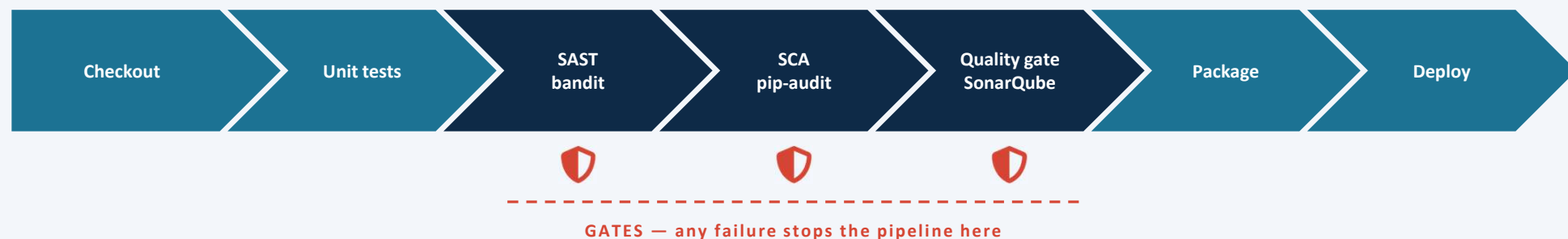
- 1 Manage Jenkins → System
- 2 Global Trusted Pipeline Libraries → Add
- 3 Name: jenkins-shared-lib · Default version: main
- 4 Retrieval: Modern SCM → Git → repo URL → Save

```
Jenkinsfile (any repo)

@Library('jenkins-shared-lib') _

pipeline {
    agent any
    stages {
        stage('Deploy') {
            steps { filDeploy('account-service', 'staging') }
        }
    }
}
```

Where quality & security gates sit



Static analysis (SAST)

Scans your code for insecure patterns — eval(), shell=True, weak crypto.

TRY LOCALLY: bandit



Dependency scan (SCA)

Checks every library in requirements.txt against known CVEs.

TRY LOCALLY: pip-audit



Quality gate

Bugs, coverage and code smells vs. the team's quality bar (SonarQube).

FIL SERVER



Change approval

An approved ServiceNow / Jira change is required before production.

FIL PROCESS

Add a Python security-scan stage

DO THIS

Add this stage after 'Test' in account-service, push, Build Now.

10 min

```
Jenkinsfile - new stage

stage('Security Scan') {
  steps {
    sh '''
      . .venv/bin/activate
      pip install --no-cache-dir bandit pip-audit
      bandit -r . -x ./venv -ll          # SAST: medium+ fails
      pip-audit -r requirements.txt      # SCA: known CVEs fail
    '''
  }
}
```

```
Console Output - expected

+ bandit -r . -x ./venv -ll
Test results:
    No issues identified.
+ pip-audit -r requirements.txt
No known vulnerabilities found
Finished: SUCCESS
```

? QUICK CHECK

Add `def calc(expr): return eval(expr)` to `account.py`, push and rebuild. What happens?

Answer: bandit flags B307 (use of eval) as MEDIUM → the stage fails → nothing ships. Remove it and you're green again.

```
SonarQube quality gate

// On FIL's Jenkins - awareness only
withSonarQubeEnv('SonarQube') { // name may differ
  sh 'sonar-scanner'
}
timeout(time: 10, unit: 'MINUTES') {
  waitForQualityGate abortPipeline: true
}
```

! WATCH OUT

If pip-audit reports a CVE, that's the gate working: upgrade the package, push, re-run.



MODULE 8

Capstone

Put it all together: ship a Python banking API through commit → acceptance → release.

IN THIS MODULE

- ✓ The mission & the architecture
- ✓ Acceptance criteria
- ✓ App files
- ✓ The Jenkinsfile, in two parts
- ✓ Run it, approve it, verify it

08

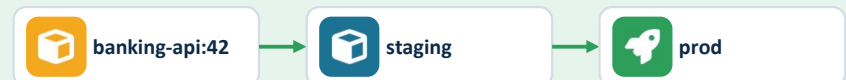
Ship a Personal Banking API

What you'll build

-  A FastAPI service with `/health` and `/balance/{id}`.
-  Tests that run in a clean Python container.
-  A Docker image, deployed to staging, then prod.
-  One human approval before production.

Key principle: Build Once, Deploy Many

- Build ONE image, tagged with the build number.
- Promote that exact image: staging → production.
- Never rebuild per environment — what you tested is what ships.
- Only config changes per environment (env vars, secrets).

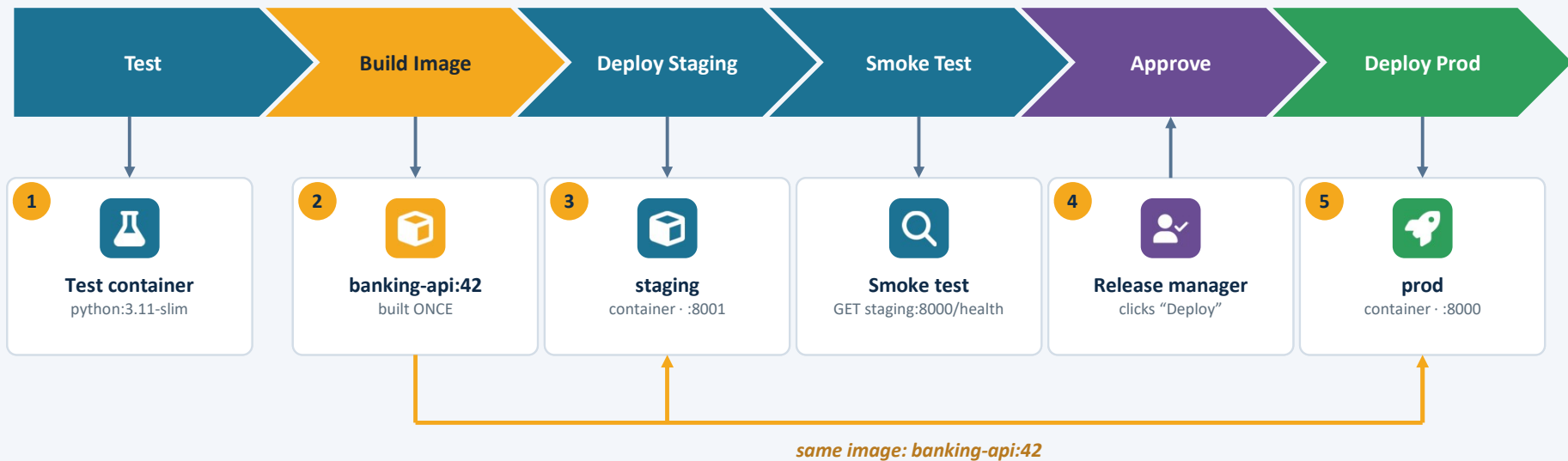


1 Docker image, built once

2 environments: staging & prod

1 human approval before prod

Build once, deploy many — the whole release



- 1 Tests run in a clean python:3.11-slim container.
- 2 The image is built ONCE and tagged with the build number.
- 3 That image runs as "staging"; a smoke test calls /health.
- 4 A human approves the release in Jenkins.
- 5 The SAME image is promoted to "prod" — nothing is rebuilt.

What “done” looks like

PART 1 · CI

Commit pipeline

- Checkout the repo
- Create a venv and install requirements
- Run pytest in python:3.11-slim
- Build image banking-api:<build #>

✓ **Verified by:** Stage View green up to Build Image

PART 2 · CD

Acceptance pipeline

- Run the image as “staging” on ci-net
- Smoke test: GET /health returns 200
- Retry while the app boots

✓ **Verified by:** localhost:8001/health answers

PART 3 · RELEASE

Release pipeline

- Pause for approval (15-min timeout)
- Deploy the SAME image as “prod”
- Re-runs replace old containers

✓ **Verified by:** docker ps: one image tag in both

Goal: one Jenkinsfile, all stages green end-to-end, no manual step except the single approval.

Create the banking-api repository

DO THIS

New repo banking-api with these files — plus the Jenkinsfile from the next two slides.

🕒 15 min

🔴 🟡 🟢 app.py

```
# app.py - Personal Banking API
from fastapi import FastAPI, HTTPException

app = FastAPI(title="Personal Banking API")
ACCOUNTS = {"ACC1001": 25000.0, "ACC1002": 1200.5}

@app.get("/health")
def health():
    return {"status": "ok"}

@app.get("/balance/{account_id}")
def balance(account_id: str):
    if account_id not in ACCOUNTS:
        raise HTTPException(status_code=404, detail="Account not found")
    return {"account": account_id, "balance": ACCOUNTS[account_id]}
```

🔴 🟡 🟢 requirements.txt

```
# pin exact versions
# in real projects
fastapi
uvicorn
httpx
pytest
```

🔴 🟡 🟢 .dockerignore

```
.venv
__pycache__
.pytest_cache
results.xml
```

🔴 🟡 🟢 test_app.py

```
# test_app.py
from fastapi.testclient import TestClient
from app import app

client = TestClient(app)

def test_health():
    assert client.get("/health").json() == {"status": "ok"}

def test_balance():
    assert client.get("/balance/ACC1001").json()["balance"] == 25000.0

def test_unknown_account():
    assert client.get("/balance/NOPE").status_code == 404
```

🔴 🟡 🟢 Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Test in a container, then build the image once

```
Jenkinsfile (part 1)

pipeline {
  agent any
  environment {
    IMAGE = "banking-api:${env.BUILD_NUMBER}" // built ONCE
  }
  stages {
    stage('Test') {
      agent {
        docker {
          image 'python:3.11-slim'
          reuseNode true // same workspace as the pipeline
        }
      }
      steps {
        sh '''
          python -m venv --clear .venv
          . .venv/bin/activate
          pip install --no-cache-dir -r requirements.txt
          pytest -v --junitxml=results.xml
        '''
      }
      post { always { junit 'results.xml' } }
    }
    stage('Build Image') {
      steps { sh 'docker build -t $IMAGE .' }
    }
  }
  // ... part 2 continues here (CD stages) ...
}
```

DO THIS

Copy part 1, then paste part 2 (next slide) directly below it — one Jenkinsfile in banking-api.

NOTE

IMAGE is set once from the build number (e.g. banking-api:42). Every later stage reuses it.

TIP

reuseNode true runs the container on the same workspace, so Build Image sees the same files.

WATCH OUT

Single quotes in sh '... \$IMAGE ...' — the shell expands the variable at run time.

Deploy, smoke-test, approve, promote

```
Jenkinsfile (part 2)

// ... continued from part 1 (still inside stages { })
stage('Deploy Staging') {
  steps {
    sh 'docker rm -f staging || true'
    sh 'docker run -d --name staging --network ci-net -p 8001:8000 $IMAGE'
  }
}
stage('Smoke Test') {
  steps {
    retry(5) {                // the app needs a few seconds to boot
      sleep 3
      sh 'curl -fs http://staging:8000/health'
    }
  }
}
stage('Approve') {
  steps {
    timeout(time: 15, unit: 'MINUTES') {
      input message: "Deploy ${env.IMAGE} to production?", ok: 'Deploy'
    }
  }
}
stage('Deploy Prod') {
  steps {
    sh 'docker rm -f prod || true'
    sh 'docker run -d --name prod --network ci-net -p 8000:8000 $IMAGE'
  }
}
} // end stages
} // end pipeline
```

NOTE

Jenkins and the app containers share ci-net, so Jenkins reaches staging by name: `http://staging:8000`.

TIP

`retry(5)` + `sleep` beats one long sleep: the stage moves on as soon as the app answers.

NOTE

`input` pauses the run for a human; `timeout` aborts it if nobody approves within 15 minutes.

WATCH OUT

`docker rm -f ... || true` removes the previous container first, so re-runs don't fail on "name in use".

Run, approve and verify the release

1 Push banking-api

App files, .dockerignore and the Jenkinsfile.

2 New Item → banking-api → Pipeline

Pipeline script from SCM, like Lab 1 → Save → Build Now.

3 Watch Stage View

Test → Build Image → Deploy Staging → Smoke Test.

4 Approve

The run pauses. Hover the Approve stage (or open Console Output) → Deploy.

5 Verify

Run the commands on the right.

Stretch goals: add the Security Scan stage · a pollSCM trigger · a failure notification in post { }.

```
Terminal - verify

# Same image in both environments?
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Ports}}"
NAMES      IMAGE          PORTS
prod       banking-api:1  0.0.0.0:8000->8000/tcp
staging    banking-api:1  0.0.0.0:8001->8000/tcp
jenkins    fil-jenkins    0.0.0.0:8080->8080/tcp, ...

# Call the API (or open the URLs in a browser)
curl http://localhost:8001/health
{"status":"ok"}
curl http://localhost:8000/balance/ACC1001
{"account":"ACC1001","balance":25000.0}
```

✓ CHECKPOINT

prod and staging show the SAME image tag — you shipped exactly what you tested.

? QUICK CHECK

Why not simply rebuild the image for production?

Answer: a rebuild can pull newer dependencies. The tested artifact is the only one you can trust.



MODULE 9

Toolkit

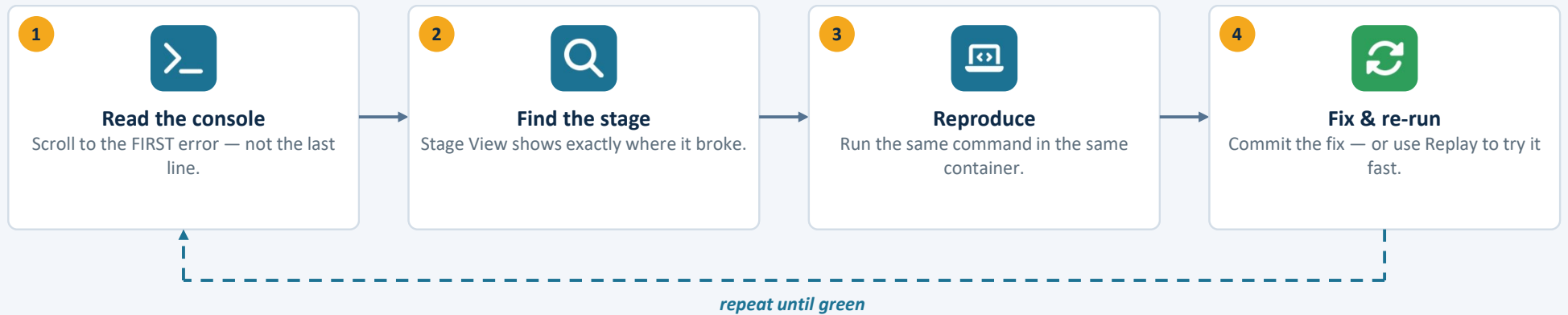
Debug like a pro, avoid the classic mistakes, and keep the essentials at hand.

09

IN THIS MODULE

- ✓ A 4-step debugging loop
- ✓ Error → fix lookup table
- ✓ Best practices
- ✓ Glossary & command cheat sheet
- ✓ Key takeaways & next steps

A 4-step debugging loop



Where to look

- Console Output
- Stage View
- Test Result page
- docker logs <container>

Power tools

- Replay — edit & re-run a build
- Pipeline Syntax — snippet generator
- cleanWs() — start from an empty workspace
- Restart from Stage (Declarative)

Asking for help

- Job name + build number
- The FIRST error line
- What changed since the last green run
- What you already tried



Common errors and how to fix them

Error you see	Likely cause	Fix
<code>Invalid agent type "docker"</code>	Docker Pipeline plugin missing	Install it — Lab 0 · Step 3
<code>permission denied ... docker.sock</code>	Jenkins user can't reach Docker	<code>docker exec -u root jenkins chmod 666 /var/run/docker.sock</code> (lab only)
<code>externally-managed-environment</code>	pip into the system Python	Use a venv: <code>python -m venv .venv</code>
<code>Permission denied: '/usr/local/lib/...'</code>	Container runs as uid 1000	Install into a venv in the workspace
<code>couldn't find remote ref .../main</code>	Branch name mismatch	Branch Specifier <code>*/main</code> (or <code>*/master</code>)
<code>Could not find credentials entry with ID</code>	Credential ID typo	Copy the ID exactly — it's case-sensitive
<code>Could not resolve host: staging</code>	Jenkins not on ci-net	<code>docker network connect ci-net jenkins</code>
<code>port is already allocated</code>	Old container still running	<code>docker rm -f prod staging</code>
<code>No such file ... requirements.txt</code>	Files not pushed / wrong Script Path	<code>git status</code> → commit → push



Best practices to carry into real projects



Keep the Jenkinsfile in Git

Pipeline changes get reviewed like code.



Fail fast

Cheap checks (lint, unit tests) first; slow ones later.



Build once, deploy many

Promote one artifact — never rebuild per environment.



Never hard-code secrets

Credentials + withCredentials. Assume logs are public.



Prefer Docker agents

Clean, reproducible builds — no snowflake machines.



Small, well-named stages

A clear Stage View makes debugging fast.



Reuse via shared libraries

Write common steps once; every team calls them.



Add timeouts

`options { timeout(...) }` stops stuck runs hogging executors.



Glossary & handy commands

Glossary

Pipeline	The whole automated process.
Stage	A named phase: Test, Build, Deploy.
Step	One command inside a stage.
Agent	Machine or container that runs stages.
Executor	A slot that runs one build at a time.
Workspace	The job's folder on the agent.
Label	Tag used to pick agents ('python').
Artifact	A build output (image, report, zip).
Credential	A secret stored & masked by Jenkins.
Webhook	Git's call to Jenkins: "new commit!"

Cheat sheet

```
# Docker
docker ps                # what's running?
docker logs -f jenkins   # watch Jenkins
docker logs staging      # app logs
docker rm -f staging prod # remove app containers
docker restart jenkins
docker exec jenkins cat \
    /var/jenkins_home/secrets/initialAdminPassword

# Handy Jenkins pages
localhost:8080/job/<job>/pipeline-syntax
localhost:8080/manage/credentials

# Clean slate - deletes ALL Jenkins data!
docker rm -f jenkins && docker volume rm jenkins_home
```



TIP

Snippet Generator: open any job → Pipeline Syntax, pick a step, fill the form, copy the generated code.

YOU DID IT

Key takeaways

- ✓ You can run Jenkins and build a real CI/CD pipeline for Python.
- ✓ A Jenkinsfile makes the pipeline versioned, reviewable, auditable.
- ✓ Docker agents give clean, repeatable builds.
- ✓ Secrets live in Credentials — never in code, never in logs.
- ✓ “Build Once, Deploy Many” is the backbone of safe banking releases.

Next steps

- Log in to FIL's Jenkins and find your squad's folder
- Add the Security Scan stage to your capstone
- Add a failure notification in post { }
- Read the Pipeline Syntax guide on jenkins.io

1 Jenkins in Docker

3 pipelines built

1 secret, used safely

1 approved release

Keep this deck — every lab is copy-paste. Come back and run them again any time.